

Описание структуры закодированного файла

Формат HUF1 для программы кодирования и декодирования по методу Хаффмана

Алгоритм	кодирование Хаффмана
Тип обрабатываемых данных	побайтовая обработка файлов
Расширение архивного файла	.huff

1. Назначение формата

Формат HUF1 используется для хранения результата кодирования файла по методу Хаффмана. Закодированный файл содержит служебную часть, необходимую для восстановления дерева Хаффмана, и непосредственно поток закодированных данных.

Кодирование выполняется побайтово: каждый байт исходного файла рассматривается как отдельный символ алфавита со значением от 0 до 255. Поэтому программа может работать не только с текстовыми файлами, но и с бинарными файлами, например с файлом Main.class.

2. Общая структура закодированного файла

Закодированный файл состоит из пяти логических частей. Служебные данные записываются в начале файла, после них располагается битовый поток, полученный в результате замены исходных байтов кодами Хаффмана.

Порядок	Часть файла	Назначение
1	Магическое число HUF1	Проверка, что файл имеет ожидаемый формат.
2	Длина исходного файла	Количество байтов в файле до кодирования.
3	Количество уникальных байтов	Размер словаря, записанного в таблице частот.
4	Таблица частот	Данные для восстановления дерева Хаффмана.
5	Закодированные данные	Поток битов, упакованный в байты.

Словарь в данном формате представлен таблицей частот. По этой таблице при декодировании заново строится дерево Хаффмана, после чего из дерева восстанавливаются коды для всех встречавшихся байтов.

3. Поля заголовка

Заголовок файла содержит фиксированную часть и таблицу частот переменного размера. Числовые значения `int` и `long` записываются в порядке байтов, используемом классами `DataOutputStream` и `DataInputStream` в Java: сначала старший байт, затем младшие байты.

Смещение	Размер	Поле	Описание
0	4 байта	HUF1	Магическое число. Используется для проверки формата при декодировании.
4	8 байт	<code>originalLength</code>	Длина исходного файла в байтах. Тип в программе: <code>long</code> .
12	4 байта	<code>uniqueCount</code>	Количество разных байтов, присутствующих в исходном файле. Тип в программе: <code>int</code> .
16	9 * N байт	Таблица частот	N записей словаря, где N равно <code>uniqueCount</code> .
16 + 9N	до конца файла	Закодированные данные	Битовый поток кодов Хаффмана, упакованный в байты.

3.1. Магическое число

Первые четыре байта файла имеют значения ASCII-символов H, U, F и 1. Это поле позволяет при декодировании отличить архив HUF1 от произвольного файла.

3.2. Длина исходного файла

Поле занимает 8 байт и хранит количество байтов в исходном файле до кодирования. Это значение необходимо, потому что последний байт закодированного потока может быть дополнен нулями справа для выравнивания до полного байта.

3.3. Количество уникальных байтов

Поле занимает 4 байта и показывает, сколько различных байтов встретилось в исходном файле. Например, для строки 1111111112222233333 значение этого поля равно 3, потому что в файле есть только байты 1, 2 и 3.

4. Таблица частот

Таблица частот является словарем закодированного файла. Она содержит только те байты, которые действительно встретились во входном файле. Каждая запись таблицы имеет фиксированный размер 9 байт.

Поле записи	Размер	Описание
value	1 байт	Значение исходного байта от 0 до 255.
frequency	8 байт	Количество появлений этого байта в исходном файле. Тип в программе: long.

Если количество уникальных байтов равно N, то в таблицу записывается N таких пар: значение байта и его частота.

Пример таблицы частот

Для исходной последовательности 1111111112222233333 таблица частот содержит следующие записи:

Байт	Символ	Частота
49	1	10
50	2	5
51	3	5

Числа 49, 50 и 51 являются десятичными ASCII-кодами символов 1, 2 и 3. По этим частотам при декодировании строится такое же дерево Хаффмана, как и при кодировании.

5. Закодированные данные

После таблицы частот записывается поток битов. Каждый байт исходного файла заменяется соответствующим кодом Хаффмана. Коды имеют переменную длину: более частые байты обычно получают более короткие коды, а менее частые - более длинные.

Так как запись в файл выполняется целыми байтами, последний байт битового потока может содержать добавочные нули. Эти нули не считаются частью исходных данных. При декодировании программа прекращает восстановление после записи количества байтов, указанного в поле originalLength.

Пример условного словаря:

байт '1' -> 0
байт '2' -> 10
байт '3' -> 11

Исходные байты заменяются последовательностью соответствующих битовых кодов.

6. Обработка специальных случаев

6.1. Пустой исходный файл

Если исходный файл пустой, в заголовке записывается длина 0 и количество уникальных байтов 0. Таблица частот и закодированный поток данных отсутствуют. При декодировании создается пустой выходной файл.

6.2. Файл с одним уникальным байтом

Если исходный файл состоит из одного повторяющегося байта, дерево Хаффмана содержит только одну вершину. В этом случае в архиве сохраняются длина исходного файла и одна запись таблицы частот. Закодированный поток битов может отсутствовать, потому что для восстановления достаточно записать этот байт нужное количество раз.

Например, для файла 1111111111 в таблице частот указывается, что байт 1 встретился 10 раз. При декодировании программа восстанавливает десять байтов со значением 1.

7. Причина хранения таблицы частот

В файле хранится не готовый набор битовых кодов, а таблица частот. Такой подход позволяет восстановить дерево Хаффмана при декодировании и получить тот же набор кодов на основе исходных частот.

Для однозначного построения дерева при равных частотах используется фиксированный порядок сравнения узлов: сначала по частоте, затем по минимальному значению байта в поддереве. Благодаря этому при кодировании и декодировании получается одинаковая структура дерева.

8. Итоговая схема формата

В общем виде структура файла HUF1 выглядит следующим образом:

4 байта	- магическое число HUF1
8 байт	- длина исходного файла originalLength
4 байта	- количество уникальных байтов N
N записей таблицы частот:	
1 байт	- значение байта value
8 байт	- частота байта frequency
далее	- закодированный поток битов, упакованный в байты

Таким образом, закодированный файл содержит всю информацию, необходимую для самостоятельного восстановления исходного файла без обращения к внешним словарям или дополнительным данным.